

Lab_Assignment_7.3

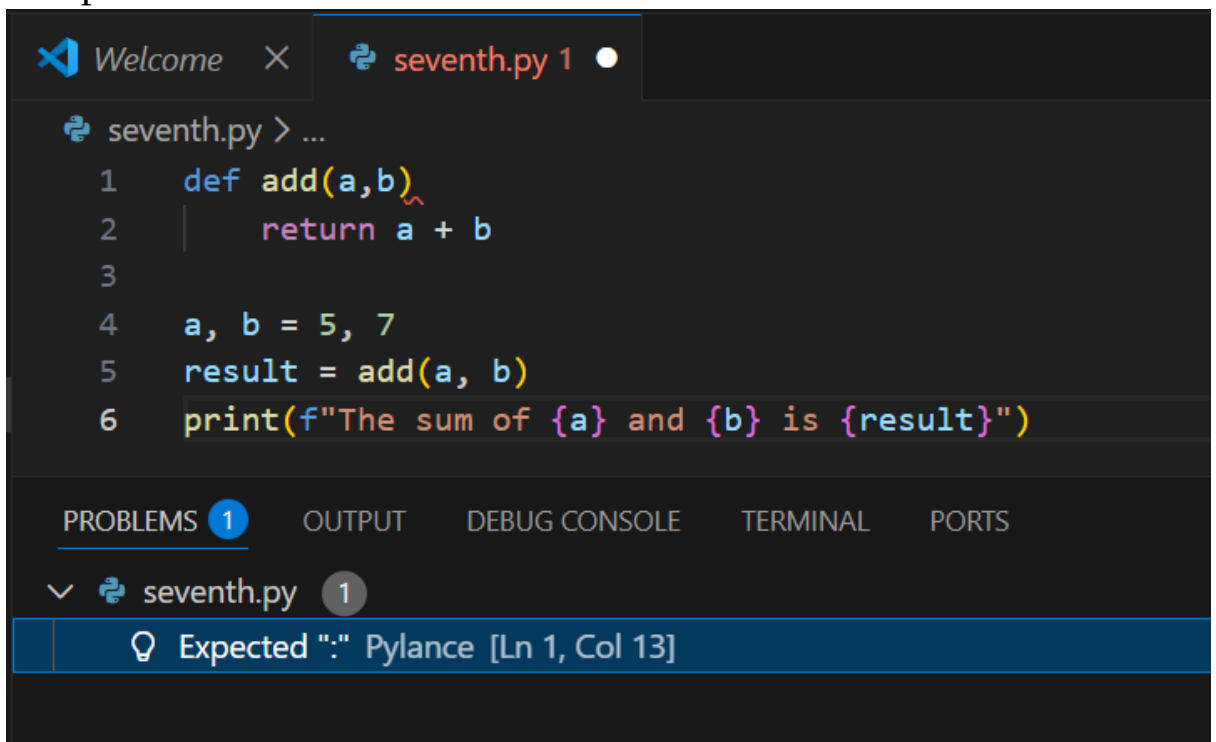
AI Assisted Coding B37 2303A52077

Task 1: Fixing Syntax Errors

- **Scenario:** You are reviewing a Python program where a basic function definition contains a syntax error.
- **Prompt:** explain the error present in the code, fix the syntax
- **Code:**

```
Def add(a,b)
    return a + b

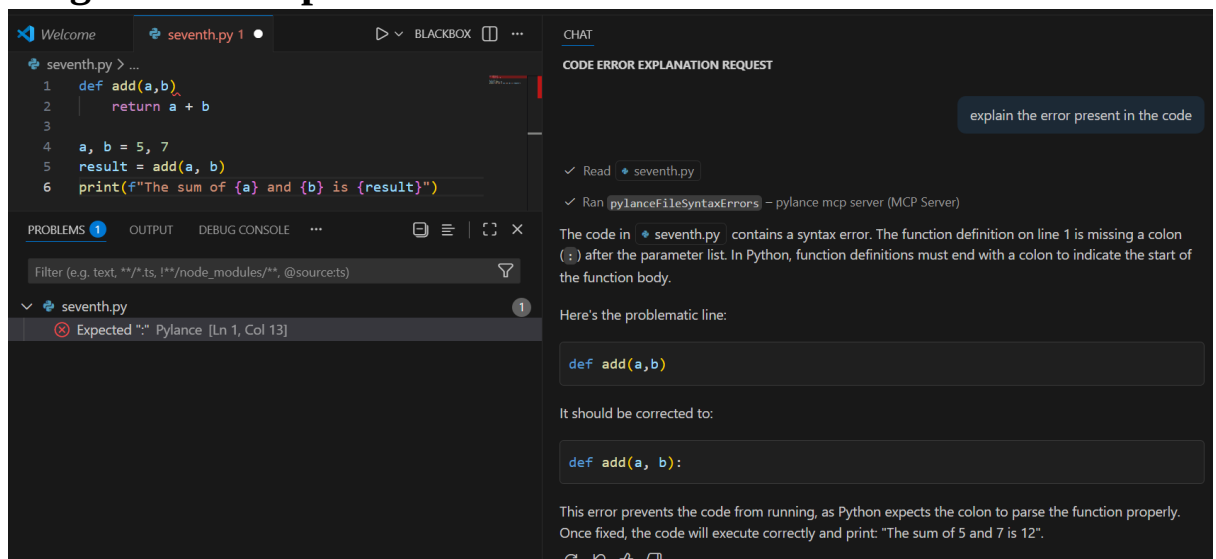
a, b = 5, 7
result = add(a, b)
print(f"The sum of {a} and {b} is {result}")
```
- **Code Explanation:** function `add`, takes two parameters `a` and `b`, and returns their sum. It is a simple mathematical function that performs addition.



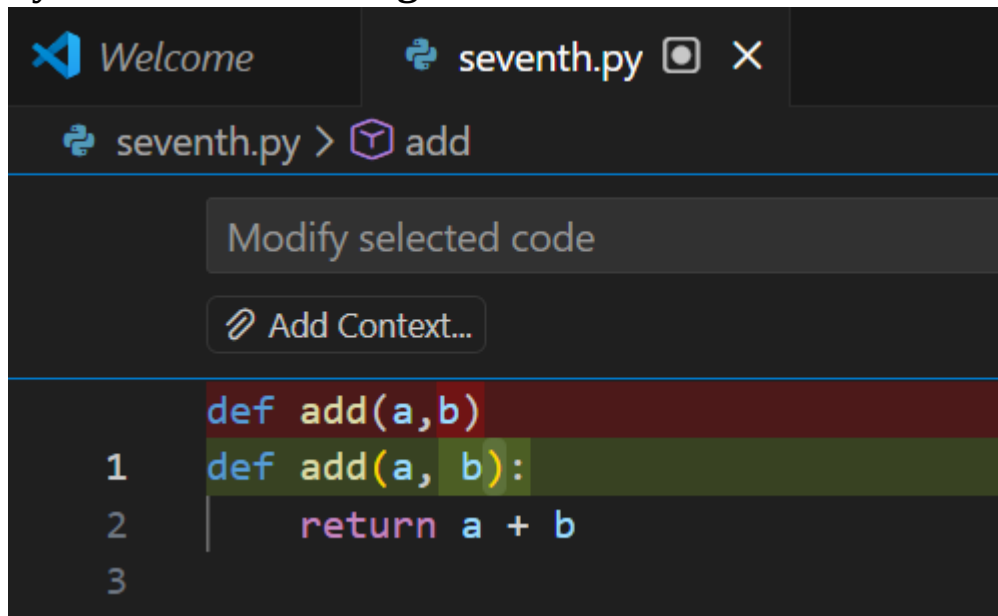
```
Welcome x seventh.py 1
seventh.py > ...
1 def add(a,b)
2     return a + b
3
4 a, b = 5, 7
5 result = add(a, b)
6 print(f"The sum of {a} and {b} is {result}")

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
seventh.py 1
Expected ":" Pylance [Ln 1, Col 13]
```

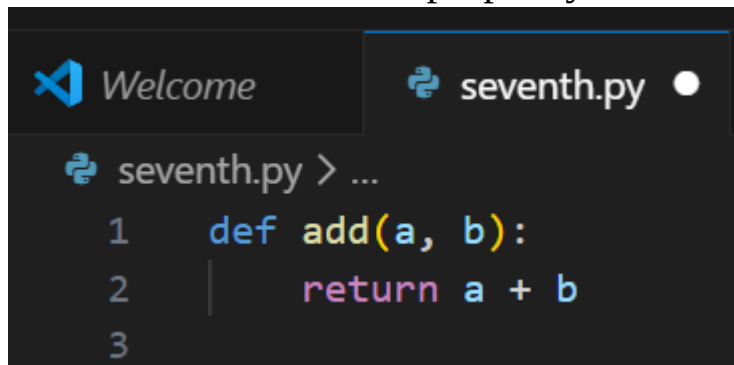
- **AI-generated explanation of the fix**



- **Syntax error resolving:**



- **Corrected function with proper syntax / Output:**



Def add(a, b):
 return a + b

Task 2: Debugging Logic Errors in Loops

- **Scenario:** You are debugging a loop that runs infinitely due to a logical mistake.

- **Code:**

```
def count_down(n):  
    while n >= 0:  
        print(n)  
        n += 1
```

- **AI explanation of the logic error**

The screenshot shows a code editor on the left and an AI chat interface on the right. The code editor displays a Python file named 'seventh.py' with the following code:

```
1 '''  
2 def add(a, b):  
3     return a + b  
4 '''  
5   
6 def count_down(n):  
7     while n >= 0:  
8         print(n)  
9         n += 1
```

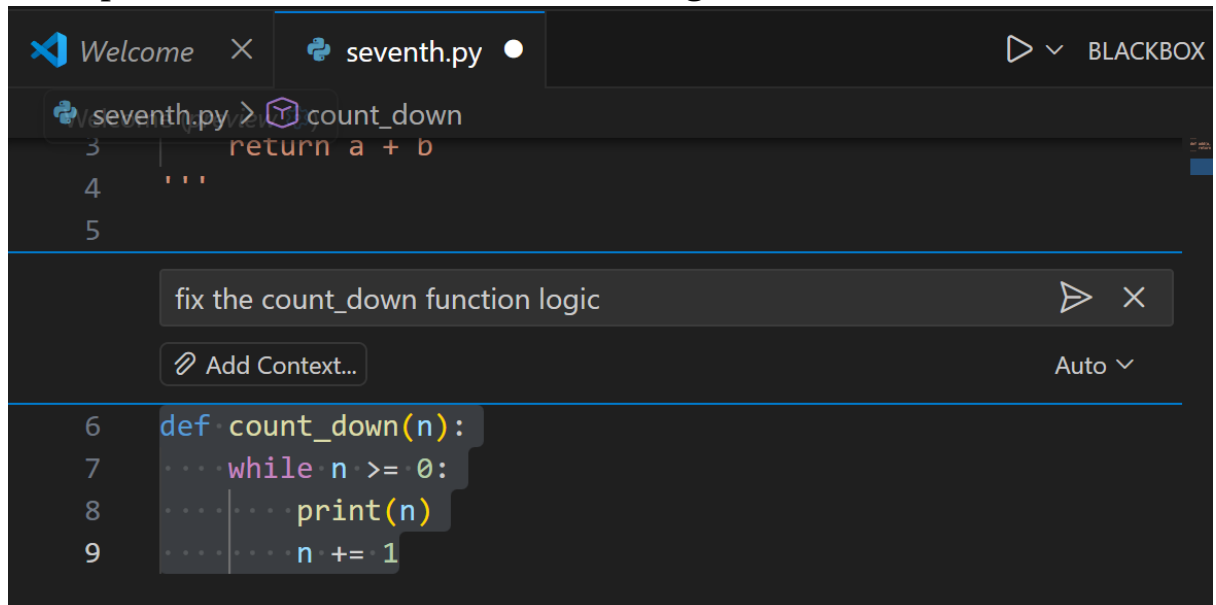
The AI chat interface on the right has a 'CHAT' tab and a 'CODE ERROR EXPLANATION REQUEST' section. A user message asks: 'do you find any issue in the logic of count_down function'. The AI response explains the logic error: 'Yes, there's a logic error in the count_down function. The function is intended to count down from the given number n to 0, but instead of decrementing n (reducing it by 1 each iteration), it's incrementing it with n += 1. This causes the loop to run indefinitely if n starts at 0 or a positive value, since n >= 0 will always remain true. To fix it, change n += 1 to n -= 1 so it properly counts down. For example, if n = 5, it should print 5, 4, 3, 2, 1, 0.'

- **Code Explanation:** function, `count\_down`, takes a single parameter `n` and prints the numbers from `n` down to 0. It uses a while loop to decrement `n` until it reaches 0, printing each

value of `n` along the way. This function is more of a procedural function that performs an action (printing)

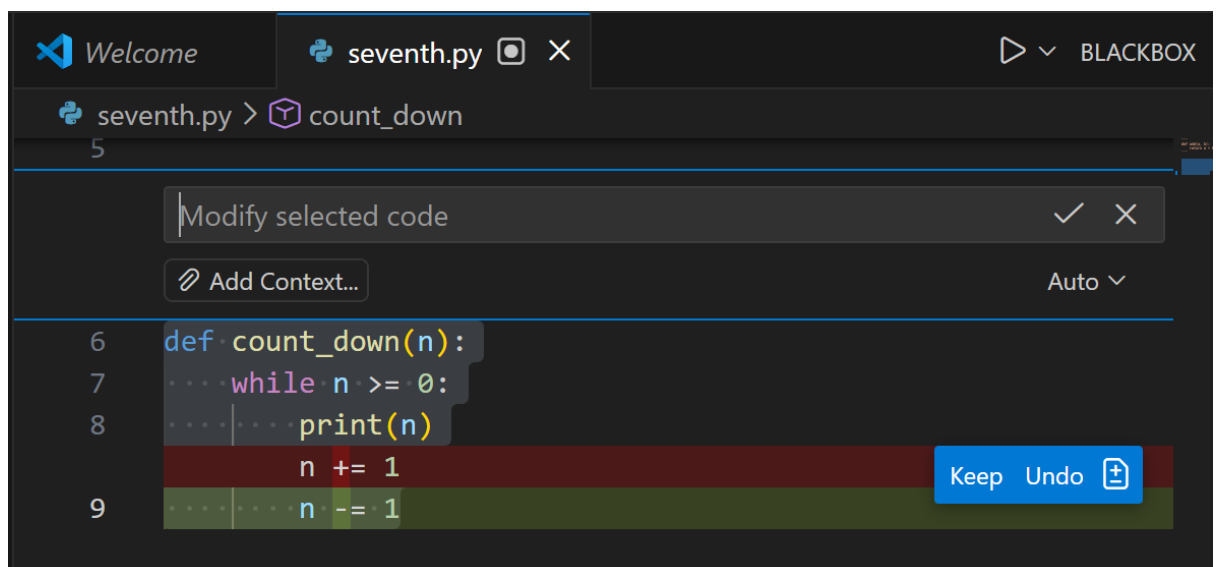
- **Debugging and fixing the code with AI:**

Prompt: fix the count_down function logic



```
seventh.py > count_down
3 |     return a + b
4 |     ...
5 |
6 | def count_down(n):
7 |     ... while n >= 0:
8 |         ... print(n)
9 |         ... n += 1
```

fix the count_down function logic



```
seventh.py > count_down
5 |
6 | def count_down(n):
7 |     ... while n >= 0:
8 |         ... print(n)
9 |         ... n -= 1
```

Modify selected code

Keep Undo

- Correct increment/decrement logic applied – **Output:**

```
• def count_down(n):
•     while n >= 0:
•         print(n)
•         n -= 1
```

```

6  ✓ def count_down(n):
7  ✓     while n >= 0:
8      print(n)
9      n -= 1

```

Task 3: Handling Runtime Errors (Division by Zero)

- **Scenario**

A Python function crashes during execution due to a division by zero error.

- **Code:**

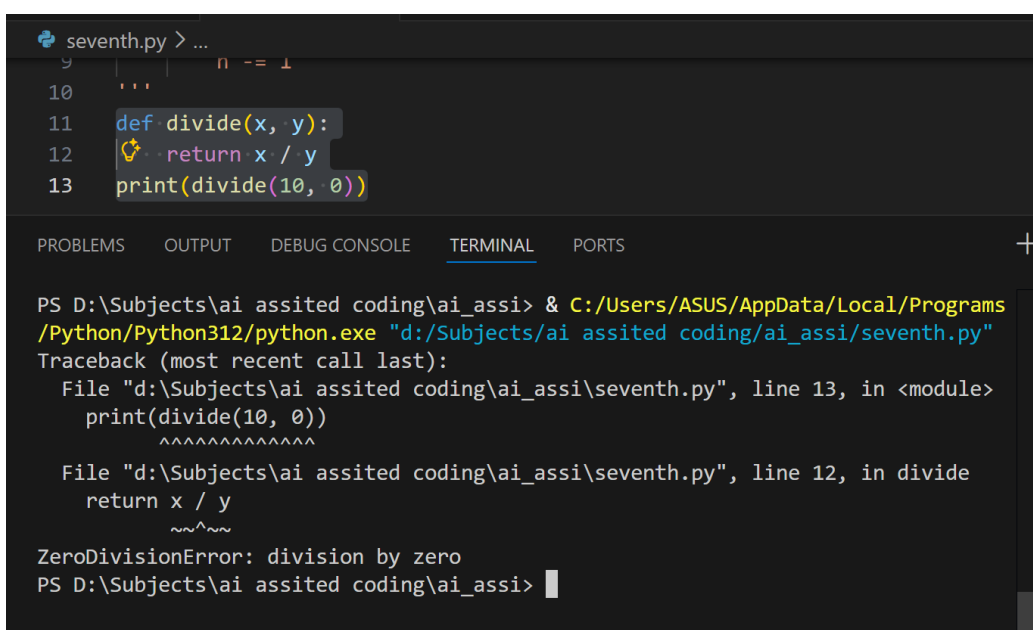
```

• def divide(x, y):
•     return x / y
• print(divide(10, 0))

```

- **runtime error:**

ZeroDivisionError: division by zero



```

seventh.py > ...
9      n -= 1
10
11  def divide(x, y):
12      return x / y
13  print(divide(10, 0))

```

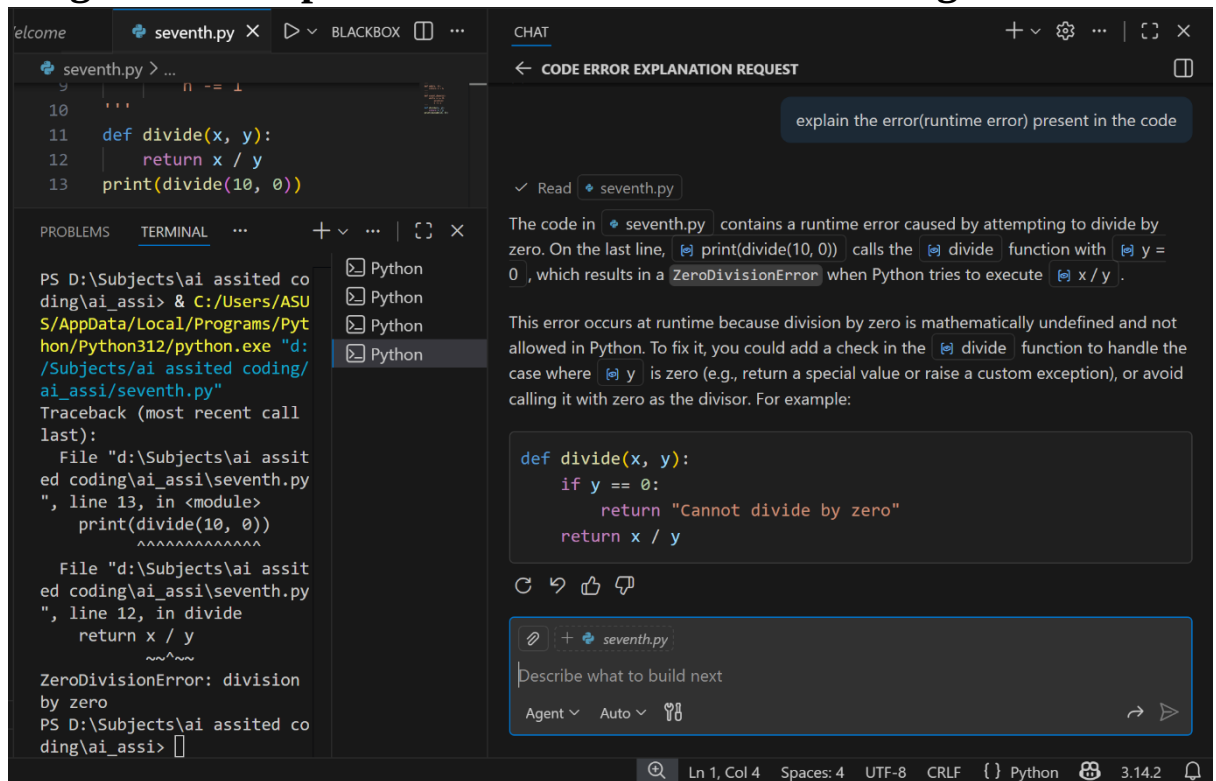
PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS/AppData/Local/Programs/Python/Python312/python.exe "d:/Subjects/ai assisted coding/ai_assi/seventh.py"
Traceback (most recent call last):
  File "d:\Subjects\ai assisted coding\ai_assi\seventh.py", line 13, in <module>
    print(divide(10, 0))
    ~~~~~
  File "d:\Subjects\ai assisted coding\ai_assi\seventh.py", line 12, in divide
    return x / y
    ~~~~~
ZeroDivisionError: division by zero
PS D:\Subjects\ai assisted coding\ai_assi>

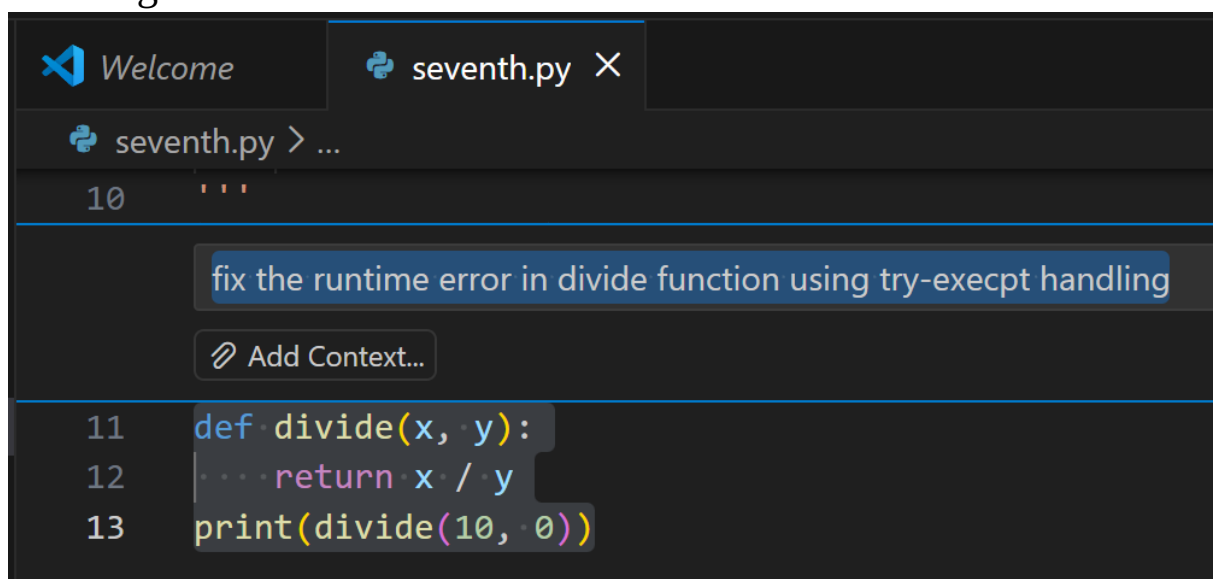
```

- **AI-generated explanation of runtime error handling**



- **Debugging and fixing the code with AI:**

Prompt : fix the runtime error in divide function using try-except handling



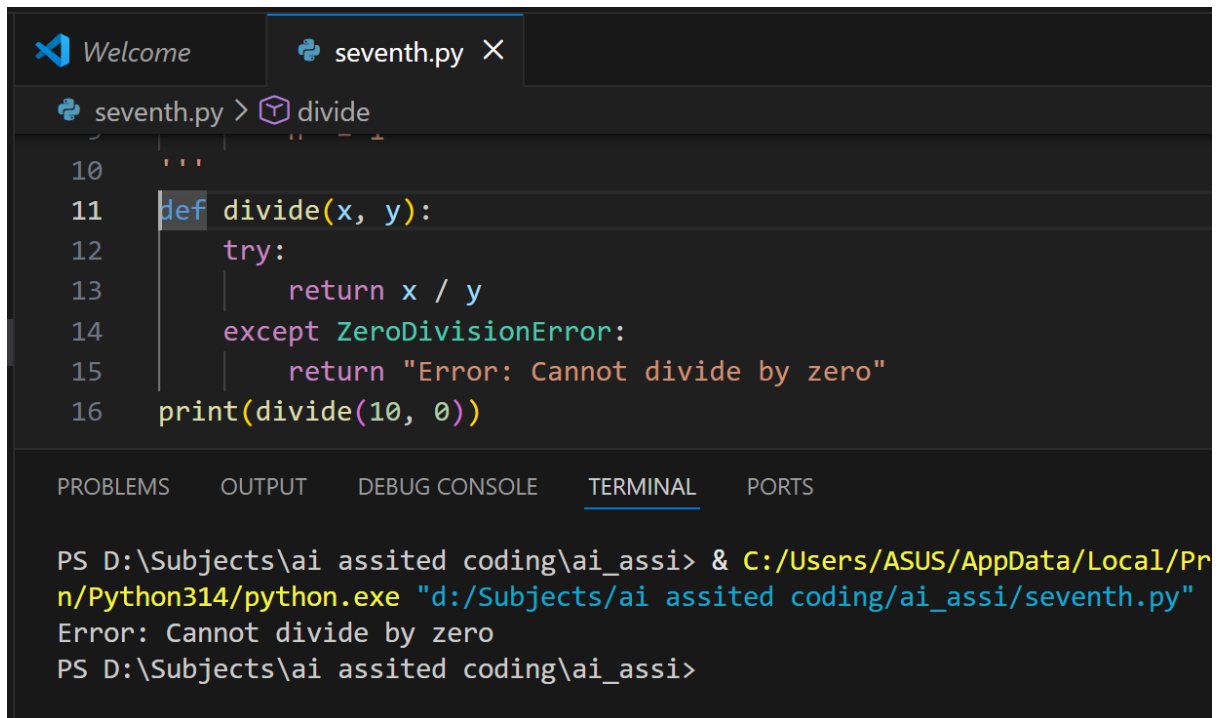
```
10 '''  
  
11 def divide(x, y):  
12     return x / y  
13     try:  
14         return x / y  
15     except ZeroDivisionError:  
16         return "Error: Cannot divide by zero"  
17 print(divide(10, 0))
```

- Final code:

```
• def divide(x, y):  
•     try:  
•         return x / y  
•     except ZeroDivisionError:  
•         return "Error: Cannot divide by zero"  
• print(divide(10, 0))
```

- **Code Explanation:** The code defines a function called `divide` that takes two parameters, `x` and `y`. It attempts to divide `x` by `y` using a try-except block. If the division is successful, it returns the result. However, if a `ZeroDivisionError` occurs (which happens when `y` is zero), it catches the exception and returns an error message stating "Error: Cannot divide by zero". Finally, the code calls the `divide` function with the arguments 10 and 0, which will trigger the error handling and return the error message.

Output:



```
10
11 def divide(x, y):
12     try:
13         return x / y
14     except ZeroDivisionError:
15         return "Error: Cannot divide by zero"
16 print(divide(10, 0))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "d:/Subjects/ai assisted coding/ai_assi/seventh.py"
Error: Cannot divide by zero
PS D:\Subjects\ai assisted coding\ai_assi>
```

Task 4: Debugging Class Definition Errors

Scenario:

You are given a faulty Python class where the constructor is incorrectly defined

Initial Code:

```
class rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Error:


```
18 '''
19 class rectangle:
20     def __init__(length, width):
21         self.length = length
22         self.width = width
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g)

✓ seventh.py 2

- ⚠ "self" is not defined Pylance(reportUndefinedVariable) [Ln 21, Col 9]
- ⚠ "self" is not defined Pylance(reportUndefinedVariable) [Ln 22, Col 9]

AI explanation of object-oriented error:

Prompt: in the class rectangle identify the error, type of error and suggest codes fixes

seventh.py 2

```
15 except ZeroDivisionError:
16     return "Error: Cannot di
17 print(divide(10, 0))
18 '''
19 class rectangle:
20     def __init__(length, width):
21         self.length = length
22         self.width = width
```

PROBLEMS 2 OUTPUT ...

Filter (e.g. text, **/*.ts, !**/node_modules/**, @source:...

✓ seventh.py 2

- ⚠ " Pylance(reportUndefinedVariable) [Ln 21, Col 9]
- ⚠ " Pylance(reportUndefinedVariable) [Ln 22, Col 9]

CHAT

← CODE ERROR EXPLANATION REQUEST

in the class rectangle identify the error, type of error and suggest codes fixes

✓ Read Python file

The `rectangle` class in `seventh.py` has an error in its `__init__` method definition.

Identified Error

- **Issue:** The `__init__` method is missing the required `self` parameter as the first argument. It currently reads `def __init__(length, width):`, but in Python, instance methods must have `self` as the first parameter to refer to the instance being created.
- **Type of Error:** This is a **runtime error** (specifically a `TypeError`). It won't cause a syntax error during parsing, but when you try to instantiate the class (e.g., `rect = rectangle(5, 10)`), Python will raise: `TypeError: rectangle.__init__() takes 2 positional arguments but 3 were given` (because Python automatically passes the instance as the first argument, making it 3 total, but the method only expects 2).

Suggested Code Fixes

To resolve this, update the `__init__` method to include `self` as the first parameter. Here's the corrected version:

```
def __init__(self, length, width):
```

Describe what to build next

Debugging and fixing the code with AI:

```
class rectangle:
    def __init__(length, width):
```

Quick Fix

- 💡 Change spelling to "Self"
- 💡 Search for additional import matches
- 💡 Add '# type: ignore' to suppress warning
- 💡 Add '# pyright: ignore[reportUndefinedVar
- 💡 Add '# type: ignore' to suppress warning
- 💡 Add '# pyright: ignore[reportUndefinedVar

🔧 Fix Enter to Apply

🔧 Explain

VS Code interface showing the file `seventh.py` and the breadcrumb `seventh.py > rectangle > __init__`.

Below the breadcrumb, there is a search bar with the text "Modify selected code" and a button "Add Context...".

The code editor shows the following code:

```
19 class rectangle:
20     def __init__(length, width):
21         ...
22         self.length = length
23         self.width = width
```

The code is highlighted with a blue selection bar, and the line numbers 19, 20, 21, and 22 are visible on the left.

Final Code:

```
class rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

Code Explanation:

The code defines a class named `rectangle` that represents a rectangle shape. The class has an `\_\_init\_\_` method, which is a special method in Python used to initialize new objects of the class. When a new instance of the `rectangle` class is created, the `\_\_init\_\_` method is called with two parameters: `length` and `width`. These parameters are used to set the instance variables `self.length` and `self.width`, which store the dimensions of the rectangle. This allows for the creation of rectangle objects with specific lengths and widths.

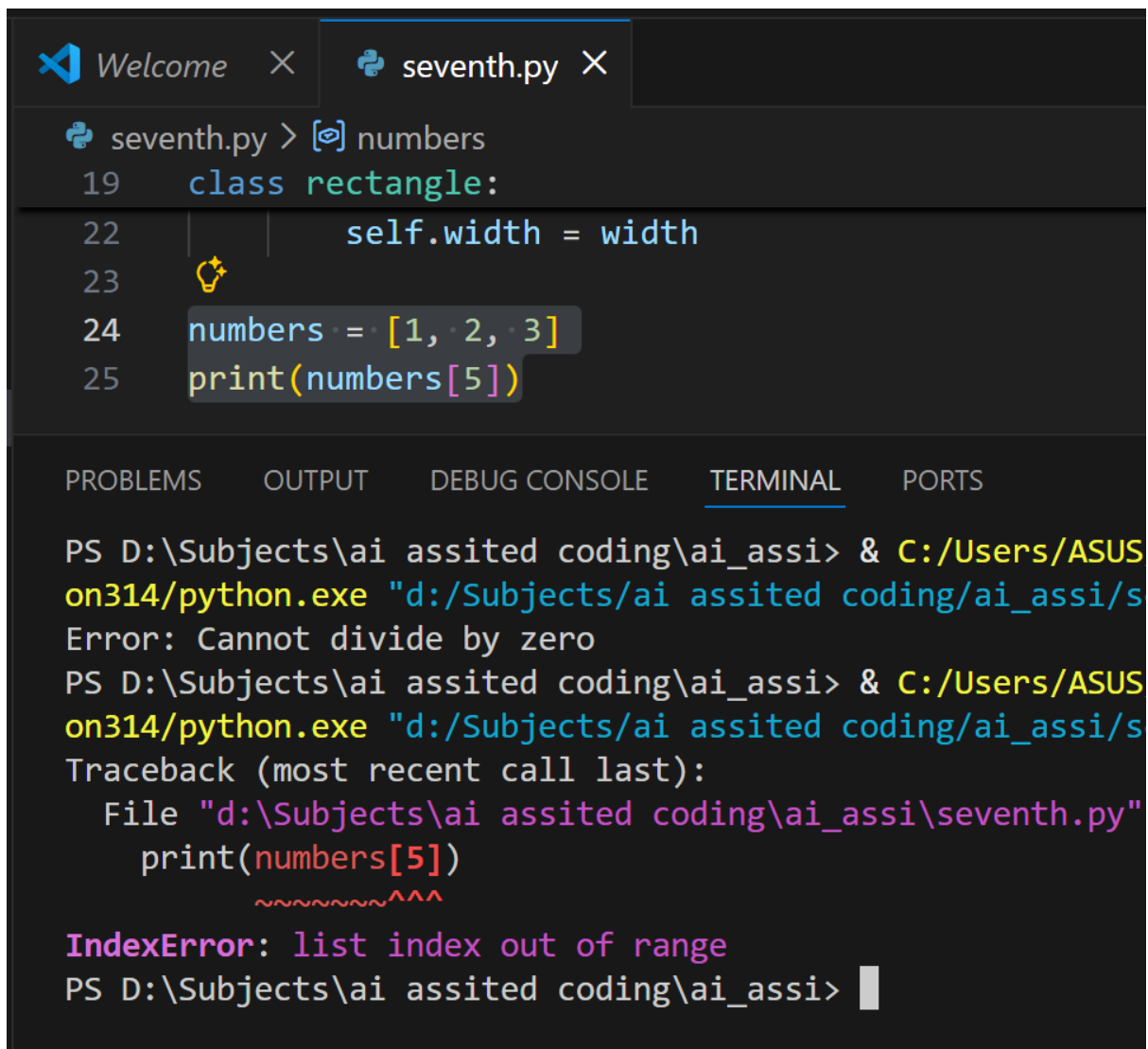
Task 5: Resolving Index Errors in Lists**Scenario:**

A program crashes when accessing an invalid index in a list.

Initial Code:

```
numbers = [1, 2, 3]
print(numbers[5])
```

Runtime(Index) Error:



The image shows a Visual Studio Code window with a file named `seventh.py` open. The editor displays the following Python code:

```
19 class rectangle:
22     self.width = width
23
24 numbers = [1, 2, 3]
25 print(numbers[5])
```

Below the editor, the **TERMINAL** tab is active, showing the command prompt output:

```
PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS
on314/python.exe "d:/Subjects/ai assisted coding/ai_assi/s
Error: Cannot divide by zero
PS D:\Subjects\ai assisted coding\ai_assi> & C:/Users/ASUS
on314/python.exe "d:/Subjects/ai assisted coding/ai_assi/s
Traceback (most recent call last):
  File "d:\Subjects\ai assisted coding\ai_assi\seventh.py"
    print(numbers[5])
          ~~~~~^^^
IndexError: list index out of range
PS D:\Subjects\ai assisted coding\ai_assi>
```

AI suggestion using length checks or exception handling:

Prompt:

suggest the ways from which we can fix Runtime(Index) Error in the code

CHAT

CODE ERROR EXPLANATION REQUEST

suggest the ways from which we can fix Runtime(Index) Error in the code

✓ Read Python file

The code in `seventh.py` contains a runtime `IndexError` on the last line:
`print(numbers[5])`. This occurs because the list `numbers = [1, 2, 3]` has only 3 elements (indices 0, 1, 2), so accessing index 5 is out of range.

Here are several ways to fix this error, depending on the intended behavior (e.g., preventing crashes, providing defaults, or validating input). I'll suggest code fixes with explanations:

1. Bounds Checking (Preventive Check)

- **Description:** Before accessing the index, check if it's within the valid range using `len(list)`. This avoids the error entirely.
- **Code Fix:**

```
numbers = [1, 2, 3]
index = 5
if index < len(numbers):
```

`seventh.py`

Describe what to build next

Agent ▾

Auto ▾



CHAT

CODE ERROR EXPLANATION REQUEST

```
index = 5
if index < len(numbers):
    print(numbers[index])
else:
    print("Index out of range")
```

- **Pros:** Simple, no exceptions raised. Good for predictable code.
- **Cons:** Requires manual checks for each access.

2. Exception Handling (Try-Except Block)

- **Description:** Wrap the access in a `try-except` block to catch the `IndexError` and handle it gracefully (e.g., print a message or return a default value).
- **Code Fix:**

```
numbers = [1, 2, 3]
try:
    print(numbers[5])
except IndexError:
    print("Index out of range")
```

`seventh.py`

Describe what to build next

CHAT

CODE ERROR EXPLANATION REQUEST

3. Use a Safe Access Function or Method

- **Description:** Create a helper function that checks bounds and returns a default value if out of range. Alternatively, use list slicing or other safe methods if applicable.
- **Code Fix** (with a helper function):

```
def safe_get(lst, index, default=None):  
    if 0 <= index < len(lst):  
        return lst[index]  
    return default  
  
numbers = [1, 2, 3]  
print(safe_get(numbers, 5, "Index out of range")) # Prints "In
```

- **Pros:** Reusable and clean. Can be extended for more complex logic.
- **Cons:** Adds extra code if not needed elsewhere.

4. Adjust the Index or List Dynamically

- **Description:** If the index is meant to be dynamic (e.g., from user input), validate or

 +  seventh.py

Describe what to build next

Agent ▾ Auto ▾ 



4. Adjust the Index or List Dynamically

- **Description:** If the index is meant to be dynamic (e.g., from user input), validate or adjust it. Or, if the list might grow, ensure the index is valid before use.
- **Code Fix** (e.g., clamp the index to the last valid one):

```
numbers = [1, 2, 3]  
index = 5  
safe_index = min(index, len(numbers) - 1) # Clamp to max valid  
print(numbers[safe_index]) # Prints 3 (last element)
```

- **Pros:** Keeps the access simple while making it safe.
- **Cons:** Assumes you want the last element; may not fit all use cases.

Additional Tips

- **Root Cause:** Always ensure indices are within `0` to `len(list) - 1`. Use loops (e.g., `for i in range(len(numbers))`) to avoid manual indexing errors.
- **Testing:** After fixing, run the code to verify. For example, use `print(len(numbers))` to confirm list length.
- **Best Practice:** Prefer bounds checking for performance-critical code; use exceptions for robustness in unpredictable scenarios.

If this doesn't match the exact error you're seeing (e.g., if there's more code or a different context), provide more details for a more tailored fix!

Debugging and fixing the code with AI:

Prompt: update the code for Safe list access logic

```
24 '''  
    update the code for Safe list access logic  
    Add Context...  
25 numbers = [1, 2, 3]  
26 print(numbers[5])
```

```
24 '''  
    Modify selected code  
    Add Context...  
25 numbers = [1, 2, 3]  
    print(numbers[5])  
26 if len(numbers) > 5:  
27     print(numbers[5])  
28 else:  
29     print("Index out of range")
```

Final Code:

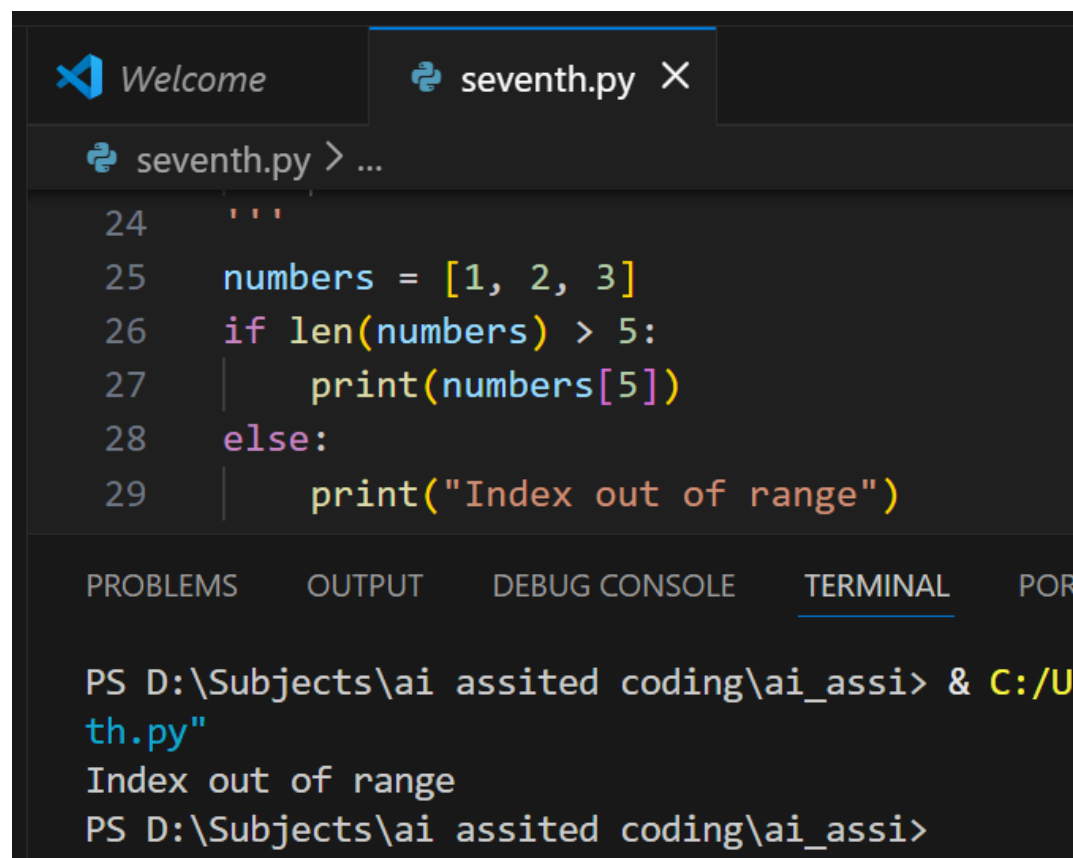
```
numbers = [1, 2, 3]  
if len(numbers) > 5:
```

```
        print(numbers[5])
else:
    print("Index out of range")
```

Code Explanation:

The code defines a list called 'numbers' containing three integers: 1, 2, and 3. It then checks if the length of the list is greater than 5. Since the length of 'numbers' is 3, which is not greater than 5, the condition evaluates to false. As a result, the code executes the else block and prints "Index out of range" to indicate that there is no index 5 in the list.

Output:



```
24 '''
25 numbers = [1, 2, 3]
26 if len(numbers) > 5:
27 |     print(numbers[5])
28 else:
29 |     print("Index out of range")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORT

```
PS D:\Subjects\ai assisted coding\ai_assi> & C:/U... th.py"
Index out of range
PS D:\Subjects\ai assisted coding\ai_assi>
```